

A Holistic Source-To-Source Compilation Approach for Partitioning and Optimizing C/C++ Applications for CPU-FPGA Systems

Tiago Santos, João M. P. Cardoso, João Bispo

Faculty of Engineering of the University of Porto, Portugal

{tiagolascasas, jmpc, jbispo}@fe.up.pt

U. PORTO
FEUP FACULDADE DE ENGENHARIA
UNIVERSIDADE DO PORTO

INESCTEC

SPeCS
Special-Purpose Computing
Systems, languages and tools

Context & Motivation

Across the computing world, from the data center to the edge, there is increasingly more interest in using heterogeneous CPU-FPGA systems to accelerate performance-critical regions of C/C++ applications.

Typically, the method of converting a generic C/C++ application to take advantage of an FPGA requires 2 separate and consecutive processes:

1. Hardware/Software Partitioning (i.e., finding the code regions to offload)
2. Optimizing the code regions for the FPGA (e.g., by using HLS directives)

There is already extensive research about each process, when looked at in isolation. But what if they could be a single process, enriched by a holistic view of the application?

We posit the question: given a heterogeneous CPU-FPGA system, does a combined partitioning and optimization scheme for an application achieve higher speedups than those achieved by applying both processes independently? **Can the whole be more than the sum of its parts?**

1. HW/SW Partitioning

```
void foo(int A[100], int B[100])
{
    for (int i = 0; i < 100; i++) {
        for (int j = 0; j < i; j++) {
            A[i] = A[i] + B[j];
        }
    }
}

void bar(int A[100], int B[100]) {
    for (int i = 0; i < 100; i++) {
        A[i] = A[i] + B[i];
    }
}
```

How do we determine the regions for offloading? Which granularity should the regions have? A function, a loop body, ...?

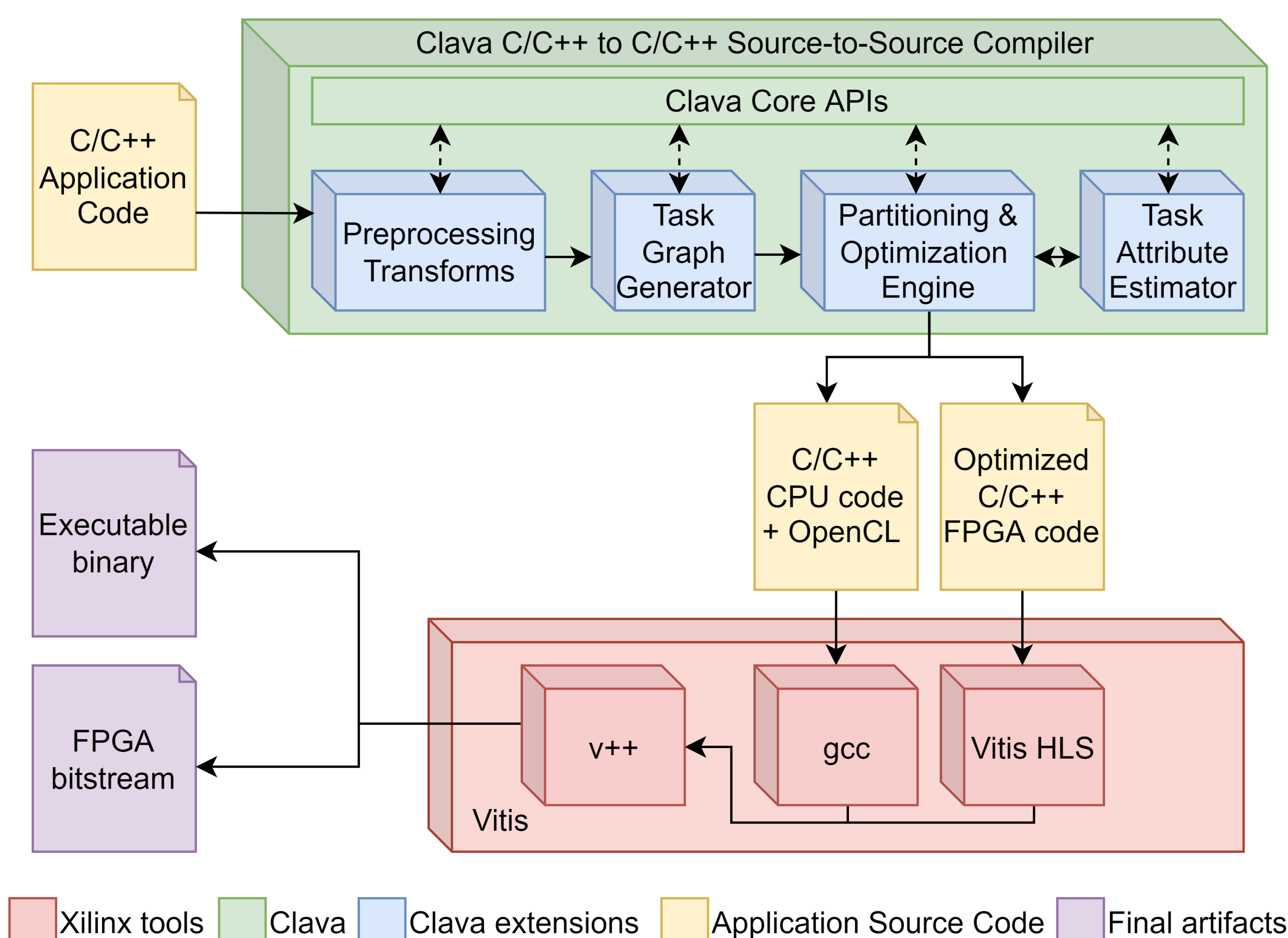
2. Code Optimizations for HLS

```
void bar(int A[100], int B[100]) {
    #pragma HLS array_partition variable=A complete
    for (int i = 0; i < 100; i++) {
        #pragma HLS unroll factor=20
        #pragma HLS pipeline
        A[i] = A[i] + B[i];
    }
}
```

Which directives should we choose? How should we configure them? Are HLS directives enough to optimize the code, or are there other code transformations we can do?

Isolated processes do not inform each other, and often require expert knowledge to work—automation is of paramount importance!

Our Toolchain

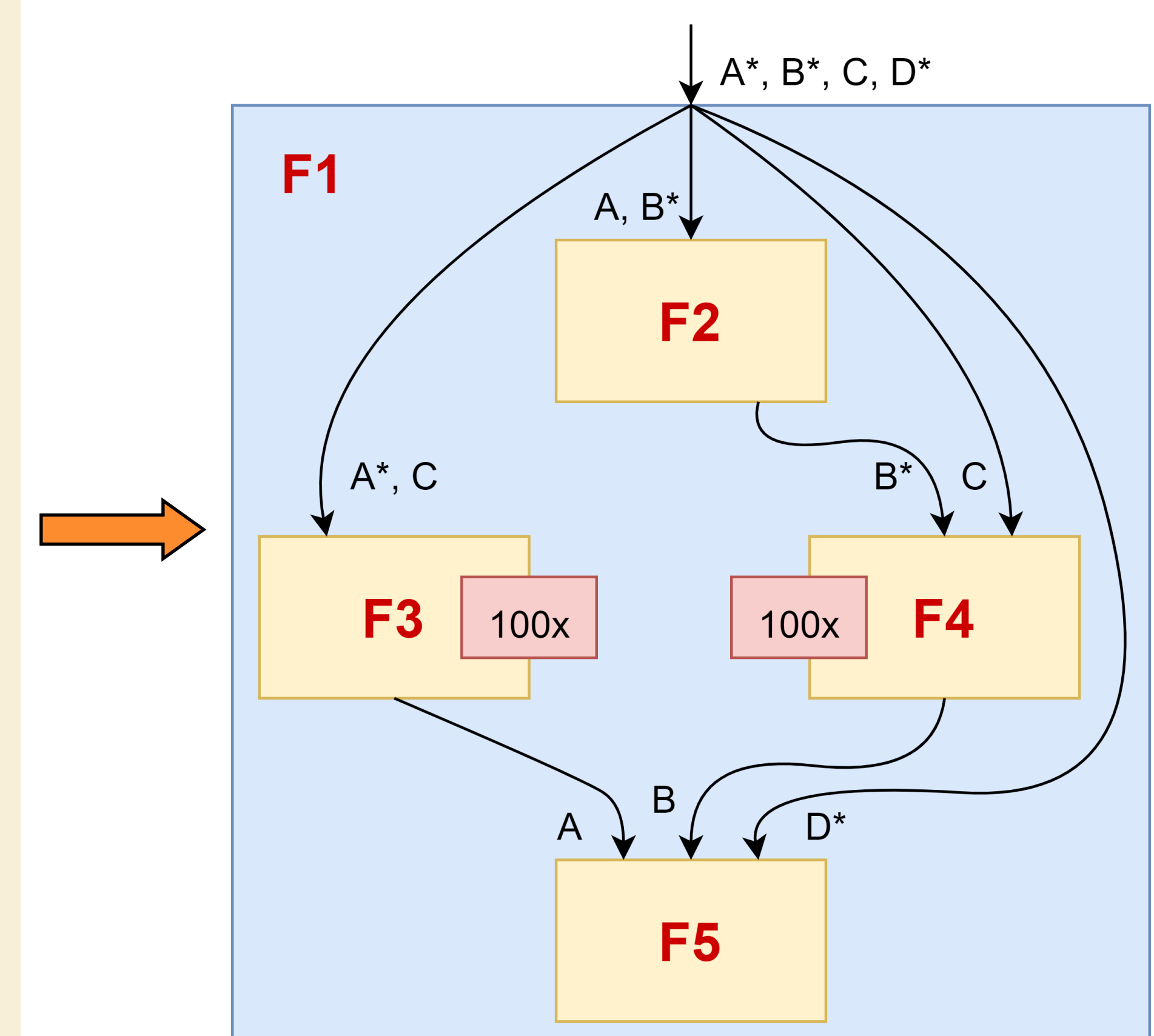


From C/C++ to a Task Graph

```
void F1(int *A, int *B, int *C, int *D)
{
    F2(A, B);
    for (int i = 0; i < 100; i++) {
        F3(A, C[i]);
        F4(B, C[i]);
    }
    F5(A, B, D);
}

void F2(int *A, int *B)
{
    for (int i = 0; i < 100; i++) {
        A[i] = A[i] + B[i];
    }
}

void F3(int *A, int c){...}
void F4(int *B, int c){...}
void F5(int *A, int *B, int *D){...}
```



The task graph is an overlay of the program's AST, with a 1:1 mapping between a task and a function, which trivializes code generation!

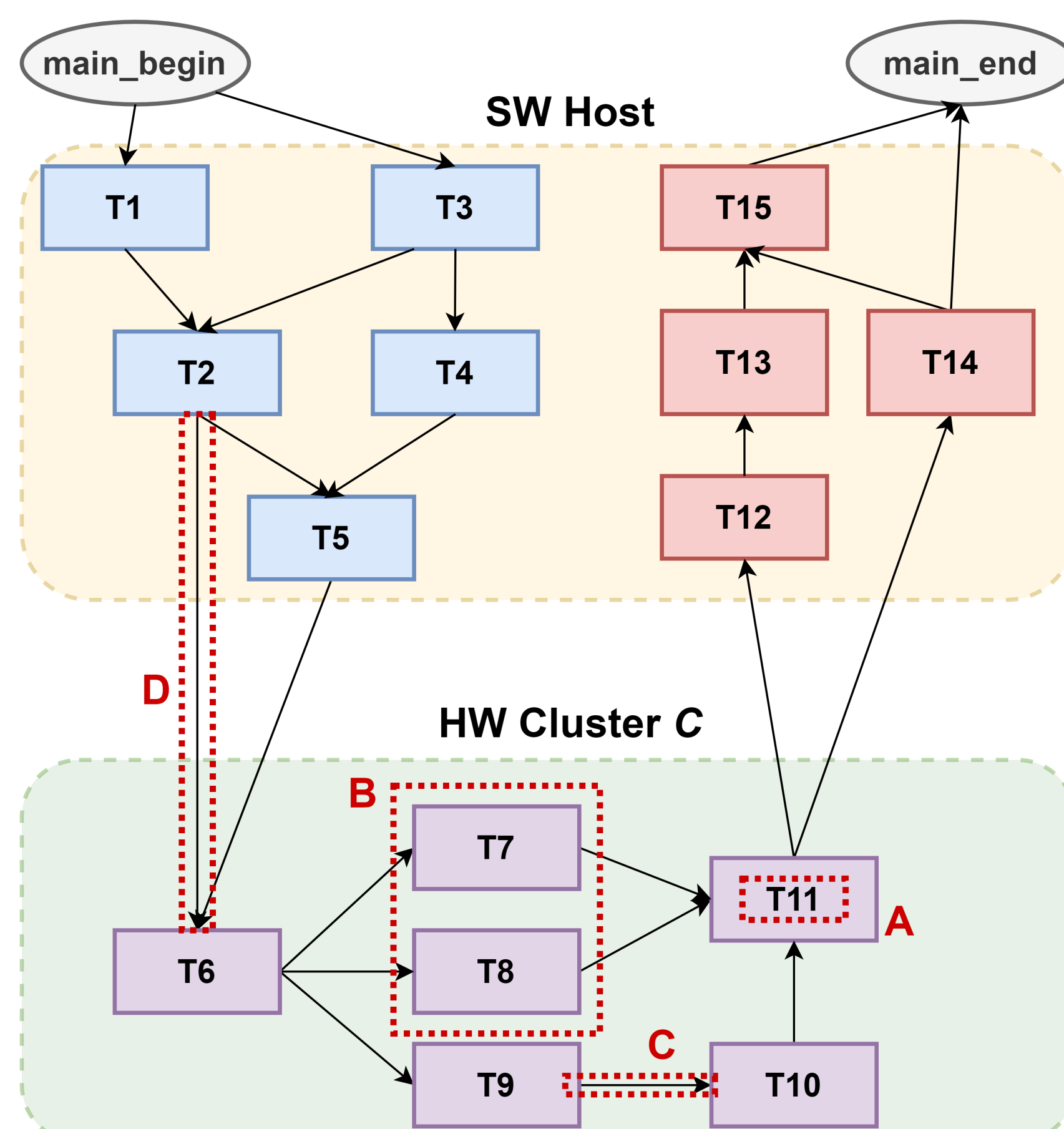
Holistic Partitioning and Optimization

Initial cluster is created using the hotspot tasks (measured through CPU profiling)

Each cluster is then expanded with "interesting" tasks from outside the cluster, such as tasks that share data

Each cluster then offers several design decisions, with optimizations enabled at multiple levels:

- A:** intra-task optim. (e.g., loop unrolling)
- B:** inter-task optim. (e.g., task fusion)
- C:** intra-cluster optim. (e.g., dataflow patterns)
- D:** inter-cluster optim. (e.g., FIFO communication from the CPU to the FPGA)



Current Progress & Future Work

Code Preprocessing Transformations **Done**

Test Environment Setup **Done**

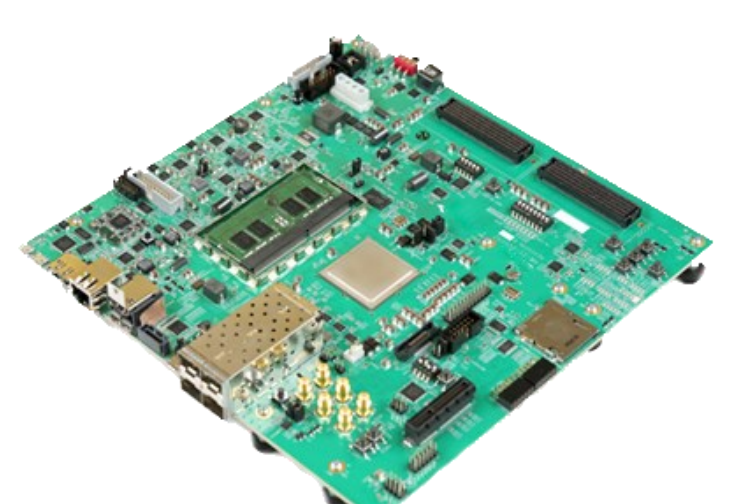
Task Graph Generation **Done**

Characterizing Task Graphs **WIP**

Generating Hotspot-based Clusters **WIP**

Generating Different Optim. Designs **TBD**

Design Selection and Refinement **TBD**



Target Evaluation Platform:
Zynq UltraScale+
MPSoC ZCU102
Evaluation Kit